

```
8 -- QUESTION 1: Reversal of Disease Signatures
9 -- Goal: Find drugs that UP-REGULATE both DDIT4 and PSME1 (to treat a deficiency).
10 SELECT m.pert_iname, AVG(s.z_score) as avg_impact
11 FROM compound_metadata m
12 JOIN genetic_signatures s
13     ON m.pert_iname = s.pert_iname
14 WHERE s.pr_gene_symbol IN ('DDIT4', 'PSME1')
15 AND s.z_score > 2.0
16 GROUP BY m.pert_iname
17 HAVING COUNT(DISTINCT s.pr_gene_symbol) = 2
18 ORDER BY avg_impact DESC
19 LIMIT 10;
```

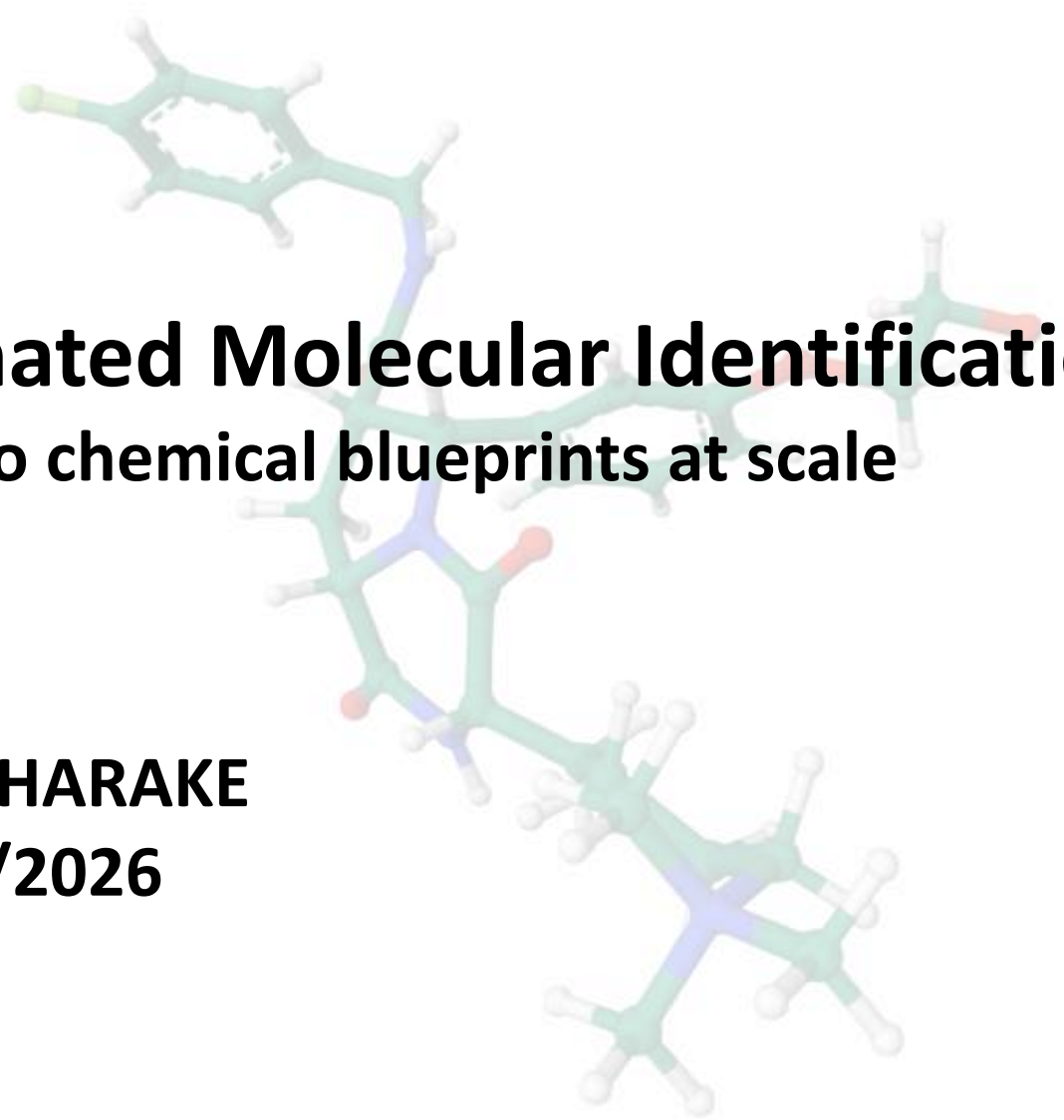
Accelerating Discovery: Automated Molecular Identification

Mapping genomic signatures to chemical blueprints at scale

```
21 -- QUESTION 2: Mechanism of Action (MoA) Discovery
22 -- Goal: Find drugs that share the same MoA as bortezomib, but do not have the same MoA as bortezomib.
23 SELECT m.pert_iname, m.moa, COUNT(*) as matching_gene_count
24 FROM compound_metadata m
25 JOIN genetic_signatures s
26     ON m.pert_iname = s.pert_iname
27 WHERE s.pr_gene_symbol IN (
28     SELECT pr_gene_symbol FROM genetic_signatures
29     WHERE pert_iname = 'bortezomib' AND z_score > 3.0
30 )
31 AND s.z_score > 3.0
32 AND m.pert_iname != 'bortezomib'
33 GROUP BY m.pert_iname, m.moa
34 ORDER BY matching_gene_count DESC
35 LIMIT 10;
```

```
37 -- QUESTION 3: Safety and Toxicity Prediction
38 -- Goal: Identify compounds with extreme transcriptomic volatility (|Z| > 4.0)
39 SELECT m.pert_iname, COUNT(*) as extreme_hit_count
40 FROM compound_metadata m
41 JOIN genetic_signatures s
42     ON m.pert_iname = s.pert_iname
43 WHERE abs(s.z_score) > 4.0
44 GROUP BY m.pert_iname
45 ORDER BY extreme_hit_count DESC
46 LIMIT 20;
```

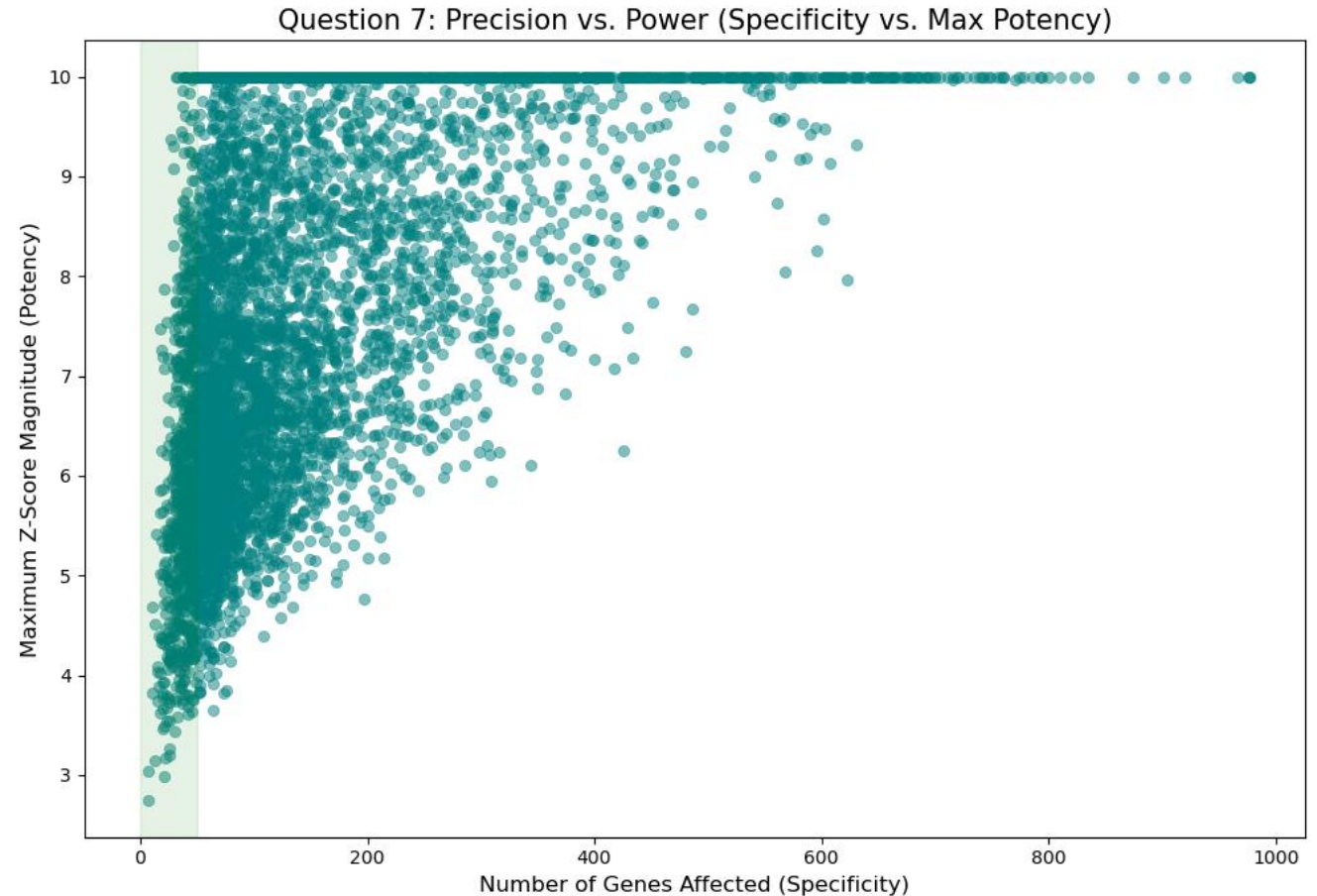
Jawad ALHARAKE
02/05/2026



Objectives

Objectives:

- Analyze millions of genetic signatures to find "Precision Candidates", molecules that surgically target and reverse disease-promoting genes.
- Build an automated web-scraping pipeline to dynamically assess the research maturity status of the top 10 drugs.



The Pipeline

The Pipeline:

1. **Phase 1 (The Surgery):** Filtered ~20GB of raw LINCS data (a massive library of cellular responses to drugs) down to **978 "Landmark"** genes for targeted dimensionality reduction.
2. **Phase 2 (The Architecture):** Engineered a Relational SQL Database (compound_metadata and genetic_signatures) with B-tree indexing for high-speed, multi-table querying.
3. **Phase 3 (The Insight):** Wrote and executed SQL queries to rank drug potency, specificity, and safety, resulting in a Top 10 Lead list.
4. **Phase 4 (Literature Search):** Engineered a custom BeautifulSoup web scraper to automatically query the US National Library of Medicine (PubMed), evaluating the research maturity and patent potential of the final leads.

Tools and Technologies

- **Python & Pandas:** The core data engineering engine used for matrix melting, data cleaning, and schema alignment.
- **SQLite3 (Relational Modeling):** Utilized to build a two-table schema and run JOIN queries.
- **Seaborn & Matplotlib:** Used to generate analytics of the business questions and publication histories.
- **Requests & BeautifulSoup:** Used to build a custom web scraper that establishes live HTTP connections with the US National Library of Medicine (PubMed).

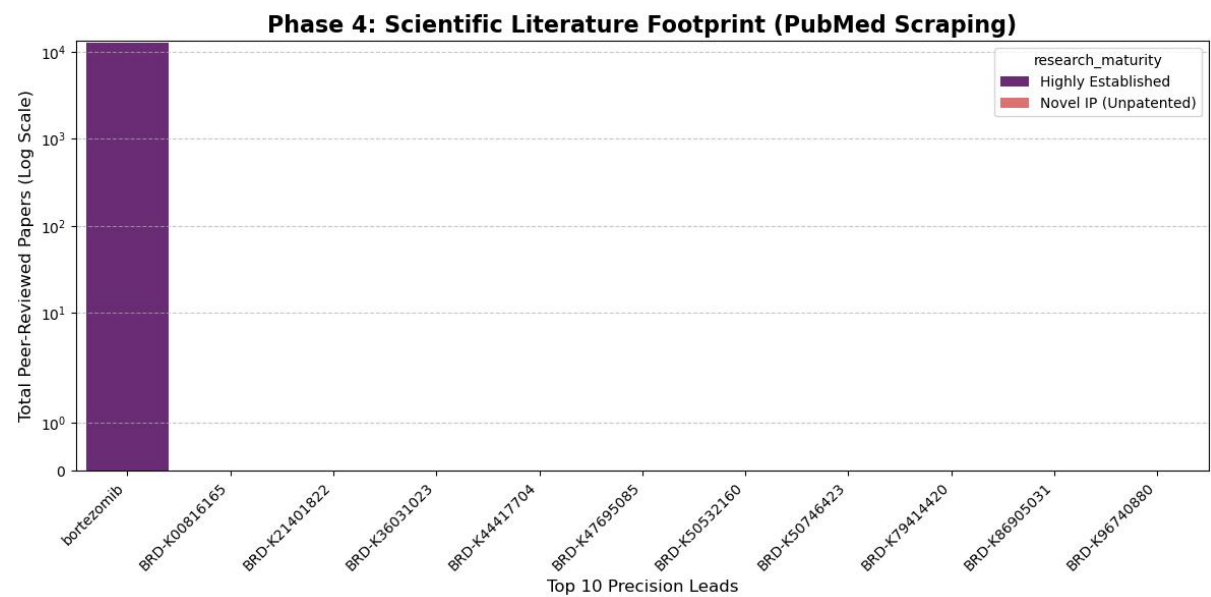
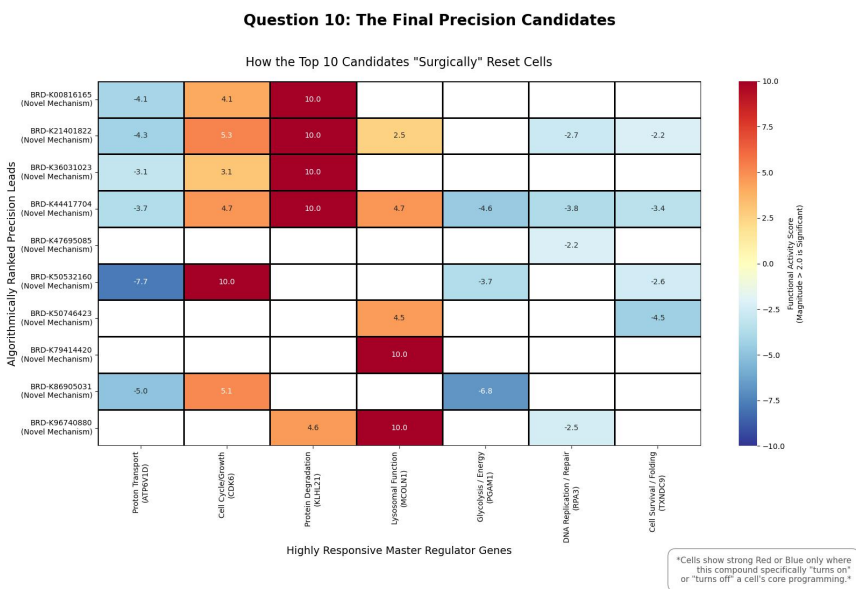


The Challenge & The Solution

- **The Challenge: Data Overload to Commercial Viability.** Needed to mine millions of raw genomic data points to find highly specific, non-toxic drug candidates, and instantly determine if the drugs were published in scientific articles or patented.
- **The Solution: Automated SQL & Web Scraping.** Architected a custom SQL database to identify the Top 10 "Precision Leads", then engineered a high-speed Python web scraper to dynamically extract their publication history from PubMed, proving their scientific novelty and commercial value.

Mission Accomplished & Future Impact

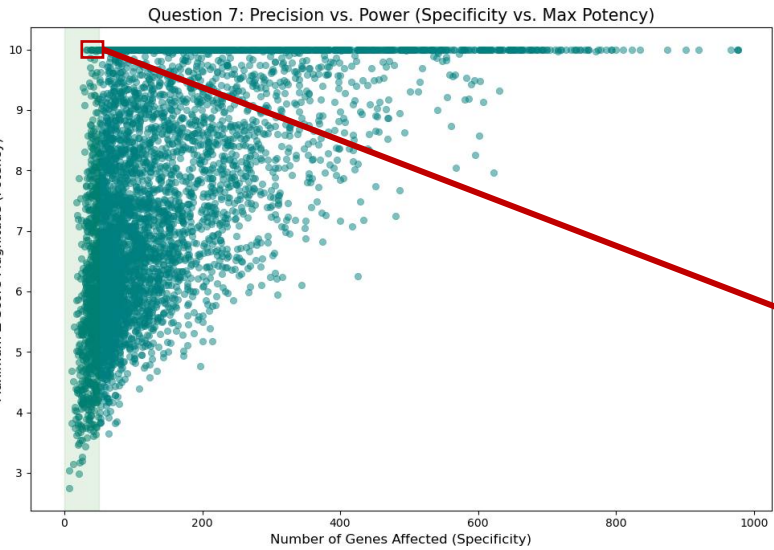
- **100% Success Rate:** Identified 10 highly potent and specific "Precision Leads" with their chemical blueprints (SMILES).
- **The Deliverable:** A curated package ranking candidates by potency and specificity.
- **100% Novelty Verified:** Unpatented Intellectual Property.
- **Future Impact:** These computationally verified, patent-ready molecular blueprints can now be exported into 3D protein-binding simulations drastically accelerating the path to clinical trials.



Thank you !
Questions and Demo...

Live Demonstration

```
SELECT m.pert_iname,
       MAX(abs(s.z_score)) as max_potency,
       COUNT(DISTINCT s.pr_gene_symbol) as affected_genes
FROM compound_metadata m
JOIN genetic_signatures s
  ON m.pert_iname = s.pert_iname
GROUP BY m.pert_iname;
```

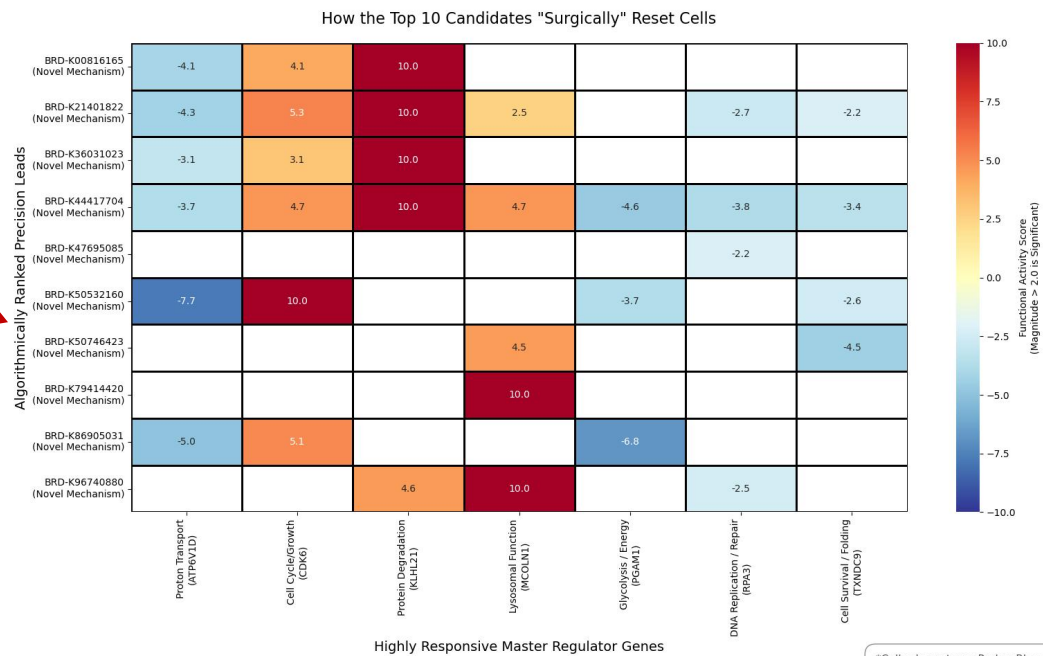


```
WITH Top10Leads AS (
  SELECT m.pert_iname
  FROM compound_metadata m
  JOIN genetic_signatures s ON m.pert_iname = s.pert_iname
  GROUP BY m.pert_iname
  HAVING COUNT(DISTINCT s.pr_gene_symbol) <= 50 AND
  MAX(abs(s.z_score)) >= 8.0
  ORDER BY MAX(abs(s.z_score)) DESC
  LIMIT 10
),
```

```
Top7Genes AS (
  SELECT s.pr_gene_symbol
  FROM genetic_signatures s
  JOIN Top10Leads t ON s.pert_iname = t.pert_iname
  WHERE abs(s.z_score) >= 2.0
  GROUP BY s.pr_gene_symbol
  ORDER BY COUNT(*) DESC
  LIMIT 7
)
```

```
SELECT m.pert_iname, m.moa, s.pr_gene_symbol, s.z_score
FROM compound_metadata m
JOIN genetic_signatures s ON m.pert_iname = s.pert_iname
JOIN Top10Leads t ON m.pert_iname = t.pert_iname
JOIN Top7Genes g ON s.pr_gene_symbol = g.pr_gene_symbol;
```

Question 10: The Final Precision Candidates



Cells show strong Red or Blue only where this compound specifically "turns on" or "turns off" a cell's core programming.

Live Demonstration

- **The Input:** top_10_lead_summary.csv (Locally sourced precision leads)
- **The Engine:** Automated PubMed Scraper (BeautifulSoup + HTTP Requests)
- **The Output:** final_executive_lead_report.csv (Investor-ready QA dataset)

pert_iname	moa	canonical_smiles	pubchem_cid	max_potency	total_gene_hits
BRD-K00816165		C[N+](C)(C)CCCC[C	24868992	10	45
BRD-K21401822		O=C(NCCc1nc2cccc	60138117	10	33
BRD-K36031023		CC(C)(C)[C@H]1C[C	16195819	10	40
BRD-K44417704		CC(=O)N1CCN(CC1,	44142151	10	40
BRD-K47695085		O=C1CCc2cc(ccc2N	60138092	10	48
BRD-K50532160		C(Nc1ncnc2cccc(cc1	44142054	10	40
BRD-K50746423		Fc1ccc(cc1)S(=O)(=O	25243714	10	39
BRD-K79414420		CCOC(=O)C1CCN(C	60138126	10	48
BRD-K86905031		Cc1cccc1CNc1ncnc	60138074	10	49
BRD-K96740880		CC(=O)N1CCN(CC1,	44142152	10	48



Step 2: The PubMed Scrapping Engine

This function sends a request to PubMed, parses the HTML, and intelligently extracts the publication count. It handles three scenarios: Multiple results, a single result, or zero results.

```
1 def scrape_pubmed_volume(drug_name):
2     print(f"🔍 Scraping PubMed Literature for: {drug_name}...")
3
4     # Format URL for PubMed search
5     url = f"https://pubmed.ncbi.nlm.nih.gov/?term={drug_name}"
6     headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'}
7
8     try:
9         # Send HTTP GET request
10        response = requests.get(url, headers=headers, timeout=10)
11        soup = BeautifulSoup(response.content, 'html.parser')
12
13        # Scenario 1: Multiple papers found (Extract the total count)
14        amount_span = soup.find('span', class_='value')
15        if amount_span:
16            count_text = amount_span.get_text().replace(',', '')
17            paper_count = int(count_text)
18            status = "Highly Established" if paper_count > 100 else "Emerging"
19            print(f"✅ Found {paper_count} published papers.")
20            return {"pert_iname": drug_name, "pubmed_papers": paper_count, "research_maturity": status}
21
22        # Scenario 2: Exactly 1 paper found (PubMed redirects to the article page directly)
23        if soup.find('div', class_='article-page'):
24            print(f"✅ Found exactly 1 published paper.")
25            return {"pert_iname": drug_name, "pubmed_papers": 1, "research_maturity": "Emerging"}
26
27        # Scenario 3: 0 papers found
28        print(f"⚠️ 0 papers found. This is an unpublished compound.")
29        return {"pert_iname": drug_name, "pubmed_papers": 0, "research_maturity": "Novel IP (Unpatented)"}
30
31    except Exception as e:
32        print(f"❌ Error scraping {drug_name}: {e}")
33        return {"pert_iname": drug_name, "pubmed_papers": 0, "research_maturity": "Error"}
```